claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-ae66ab3bc51f9a7cb.jsonl`
- SHA-256: `3a74b50b1d6fc75562a18c963ae6732d08170d6fbfa59d330fc280279d6d38a9`
- Source modified: `2026-06-18T07:54:11+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_e5140909-890`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T07:52:17.090Z)

```
Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.
Adversarial bug-hunter. Report ONLY real, reachable defects (crash on a reachable input, wrong
logic, inverted condition, bad error handling, resource leak, off-by-one, contract mismatch,
unguarded external/persisted-data access). Exact file+line + concrete trigger + minimal fix +
confidence. PRECISION over recall ? empty list is a respected answer; do NOT invent findings.
The repo has ~855 passing tests + several hardening sweeps; expect few. Skip pure style.

CANDIDATE:
{
  "title": "FusionSegmenter axis-estimate cache keyed on id(points) can return a stale play-axis
estimate across videos",
  "file": "backend/pipeline/segmenters/fusion_hybrid.py",
  "line": "100-109 (_axis_estimate, key = id(points))",
  "trigger": "Reusing one FusionSegmenter instance for two process_video calls where the second
call's `points` list is allocated at the same memory address a prior (freed) trajectory list
held ? CPython readily reuses freed list addresses. _axis_cache_key still equals id(points) so
the cached (axis, net, span) from the previous video is returned instead of re-estimating.",
  "why_wrong": "id() is reused after garbage collection, so id()-as-cache-key is unsound across
object lifetimes. A stale net/axis estimate silently mis-scores net_crossings (wrong deadband
and net position) for the new video, which can flip Gate-A keep/drop decisions. Within a single
process_video it is safe (points stays alive), so impact is limited to instance reuse.",
  "severity": "low",
  "proposed_fix": "Key the cache on a content fingerprint, pass the estimate explicitly through
the enrich call chain, or reset self._axis_cache_key/_axis_cache at the start of each
process_video; do not rely on id() identity across calls.",
  "confidence": 0.3,
  "area": "backend/pipeline/segmenters/ + stitcher/ + validators/"
}

Read C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-
base/backend/pipeline/segmenters/fusion_hybrid.py + related test. Two lenses below; you are:
REFUTE ? show it is intended/guarded/tested; default real=false unless provably a bug.
```

2. assistant (2026-06-18T07:52:19.915Z)

```
I'll analyze this candidate bug about the axis-estimate cache keyed on `id(points)`. Let me read
the actual code first.
```

3. user (2026-06-18T07:52:20.651Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
```

```
Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
shuttle-seeded window via two overridable seams the engine exposes:

- ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
  GPU-free, computed from the trajectory we already have); and, only when the person cue
  is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
  (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
  fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads the
  torchvision/supervision model ? so the default (person-OFF) path never loads a detector
  model nor touches the GPU. (torch/cv2 themselves are imported by the registry regardless,
  via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
- ``_select_for_handoff`` ? optional served Gate A/B: when ``indexing.fusion.min_crossings``
  (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows are
  dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
  **Persisted candidates remain the full superset** regardless, so the offline experiment
  harness can re-score any variant.

Defaults reproduce the substrate exactly: net_crossings/person features are persisted but
nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config seeds
and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the default
segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift through the
experiment harness (EXPERIMENT_HARNESS.md), never silently.
"""
from typing import Any, Dict, List, Optional, Tuple

from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
from backend.pipeline.detectors import rally_gate
from backend.pipeline.detectors import fusion_features as ff


class FusionSegmenter(TrajectoryHybridSegmenter):
    """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate B)."""

    family = "fusion"
    display_label = "Fusion"

    def __init__(self, db: Any, config: Any):
        super().__init__(db, config)
        # Built lazily only if the person cue is enabled (no detector model loaded otherwise).
        self._person: Optional[Any] = None
        # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-estimate it
        # over the whole trajectory once per candidate window (it depends only on `points`).
        self._axis_cache_key: Optional[int] = None
        self._axis_cache: Optional[tuple] = None

```

```
    @property
    def name(self) -> str:
        return "fusion_hybrid"

    @property
    def description(self) -> str:
        return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate rallies, "
                "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and the "
                "AI provider sets semantic boundaries.")

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
        if substrate == "tracknet":
            cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
            return TrackNetRunner(cfg), {
                "weights_path": cfg.weights_path,
                "queue_length": cfg.queue_length,
                "fusion_substrate": "tracknet",
            }
        wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate": "wasb"}

    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        stats = super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
        # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always persisted
        # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-estimated
        # over the whole trajectory by rally_gate (camera-agnostic); reused across windows.
        gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
                                        min_crossings=0, estimate=self._axis_estimate(points))
        stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

        fcfg = idx_cfg.get("fusion", {})
        if fcfg.get("cue_flags", {}).get("person", False):
            stats.update(self._person_features(cw, points, fps, frame_width, video_path, fcfg))
        return stats

    # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN + NEXT_STEPS 2026-06-14):
    # Person features (from _person_features when cue_flags.person) are now persisted for the harness
    # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation (compute only
    # for shuttle-seeded windows + padding, not whole video) is already in place via the lazy
    # self._person + windowed detections_per_frame. Next: actual harness run + lift measurement
    # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
    # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for discipline.

    def _axis_estimate(self, points: List[Any]) -> tuple:
        """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so it's
```

```
        computed once per video, not once per candidate window (it depends only on points)."""
        key = id(points)
        if self._axis_cache_key != key or self._axis_cache is None:
            self._axis_cache_key = key
            self._axis_cache = rally_gate.estimate_play_axis(points)
        est = self._axis_cache
        assert est is not None
        return est

    def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                         frame_width: float, video_path: str,
                         fcfg: Dict[str, Any]) -> Dict[str, float]:
        """GPU path (person cue ON): run the person detector over this window's ORIGINAL-video
        frame range and compute the scale/translation-invariant person features. Lazily builds
        ONE PersonDetector. ? real-video verification is owner-side; this wiring is mock-tested."""
        # Lazy import so the default (person-OFF) path never pulls torch/cv2 through this module.
        from backend.pipeline.detectors.person_detector import PersonDetector
        if self._person is None:
            self._person = PersonDetector(self.config)

        frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
        # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
        # window-edge off-by-one that drops the boundary frame from the shuttle alignment.
        start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
        det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
        fw = det["frame_width"] or frame_width
        fh = det["frame_height"]
        # If we can't trust the frame dims, every normalised feature would silently collapse
        # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked one.
        if fw <= 0 or fh <= 0:
            return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}

        poly = fcfg.get("court_polygon")
        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                          court_polygon=polygon, net=net)

    def _select_for_handoff(self, windows: List[Dict[str, Any]],
                            window_stats: List[Dict[str, Any]],
                            idx_cfg: Dict[str, Any]) -> List[int]:
        """Served Gate A/B over the (already-enriched) per-window stats. Default thresholds
        (0) keep every window ? identical to the substrate. Persisted candidates are
        untouched (full superset).

        NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If the
        thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed trajectory
        where no segment clears ``min_crossings``), the gate falls back to the full ungated
        set rather than emit a video with zero rallies. This makes Gate A/B strictly
        *pruning* ? it can shrink a candidate set but never zero it out ? so layering it on
        the substrate is "do no harm": the bounded-windowing / largest-internal-gap
```

```
split
        below it always wins on an otherwise-empty video. Observed on real footage
        (mahadevpura-1): a single mega-window split into pieces, none clearing the crossings
        gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
        fcfg = idx_cfg.get("fusion", {})
        min_crossings = int(fcfg.get("min_crossings", 0))
        min_players = int(fcfg.get("min_players", 0))
        person_on = bool(fcfg.get("cue_flags", {}).get("person", False))

        keep: List[int] = []
        for i, st in enumerate(window_stats):
            if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
                continue
            if person_on and min_players > 0 and st.get("players_in_court", 0.0) < min_players:
                continue
            keep.append(i)

        if not keep and window_stats:
            # Gating emptied a non-empty video ? fall back to the ungated set.
            return list(range(len(window_stats)))
        return keep


segmenter_registry.register(FusionSegmenter)

```

4. assistant (2026-06-18T07:52:23.559Z)

Now let me understand the key question: is `_axis_estimate` called within a single `process_video` lifecycle where `points` stays alive, or can the instance be reused across videos? Let me look at the parent class to understand the call flow.

5. user (2026-06-18T07:52:24.685Z)

```
"""Shared engine for trajectory-substrate hybrid segmenters.

tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
now the single home for the whole pipeline shape:

    healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
    -> P3 AI-provider handoff over padded clips -> P4 DB population/linking

A concrete subclass supplies only its identity and runner wiring:
- ``family``        ? config section under ``indexing.*``, output subdir, and
                      the ``<family>-hybrid-<model>`` detector tag in metadata.
- ``display_label`` ? human label for log lines ("WASB", "TrackNet").
- ``name`` / ``description`` properties (the registry contract).
- ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
                      detection_params extras such as weights_path).

This base is deliberately NOT registered; only concrete subclasses register.
"""
import os
import time
import logging
import concurrent.futures
from typing import Any, Dict, List, Optional, Tuple

import cv2

from backend.pipeline.segmenters.base import BasePlaySegmenter
from backend.utils.video import get_video_duration, extract_video_chunk
```

```python
from backend.sports import sport_registry
from backend.providers import provider_registry
from backend.pipeline.detectors.base import DetectorRunner

logger = logging.getLogger(__name__)


def _fmt_ts(seconds: float) -> str:
    seconds = max(0, int(seconds))
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    return f"{h:02d}:{m:02d}:{s:02d}"


class TrajectoryHybridSegmenter(BasePlaySegmenter):
    """Trajectory-detector hybrid: precise candidate rallies + AI semantic boundaries."""

    #: config section under `indexing` (velocity_threshold lives there), the
    #: output subdir for trajectories, and the metadata detector-tag prefix.
    family: str = ""
    #: human-readable label used in log lines.
    display_label: str = ""

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Return (runner, detector-specific detection_params extras)."""
        raise NotImplementedError

    @staticmethod
    def _probe_fps_width(video_path: str) -> tuple:
        """Return (fps, frame_width) for a video, with sensible fallbacks."""
        cap = cv2.VideoCapture(video_path)
        fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
        width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
        cap.release()
        return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

    @staticmethod
    def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
        """Provider-reported exchange count, else a duration-based estimate.

        One canonical implementation ? the twins had drifted (wasb relied on
        ``int(None)`` raising into the fallback; same result, by accident).
        """
        shot_count = segment.get("shuttle_exchange_count")
        if shot_count is None:
            return max(3, int(duration / 0.8))
        try:
            return int(shot_count)
        except (ValueError, TypeError):
            return max(3, int(duration / 0.8))

    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
```

```
        return {
            "peak_velocity": cw["peak_velocity"],
            "mean_velocity": cw["mean_velocity"],
            "active_frame_count": cw["active_frame_count"],
            "track_id_count": cw["track_id_count"],
        }

    def _select_for_handoff(self, windows: List[Dict[str, Any]],
                            window_stats: List[Dict[str, Any]],
                            idx_cfg: Dict[str, Any]) -> List[int]:
        """Indices of the candidate windows that proceed to the AI handoff (and thus may
        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
        Persisted candidates are NOT affected ? they remain the full superset for offline
        re-scoring."""
        return list(range(len(windows)))

    def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                      player_pool: Optional[List[str]] = None,
                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
        # override-ignore: the ABC declares the Result contract (Success|Failure)
        # but every live segmenter still returns a bare list ? the documented
        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
        label = self.display_label
        # --- Sport profile + AI provider wiring (identical across segmenters) ---
        sport_name = self.config.get("sport", "badminton")
        try:
            sport_profile = sport_registry.get(sport_name)
        except KeyError as e:
            logger.error(str(e))
            return []

        idx_cfg = self.config.get("indexing", {})
        # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become the
        # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key is
        # needed and nothing is uploaded. Used to run the local detector standalone (e.g. to
        # measure it against the Gemini path, or to avoid the cloud entirely). With
        # FusionSegmenter the windows are still Gate-A/B-filtered via `_select_for_handoff`.
        skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
        provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
        if skip_ai:
            model_name = "local-noai"
            logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
                        "be emitted as rallies; no %s handoff, no API key needed.",
                        label, provider_name)
        else:
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))
            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                logger.error(
                    f"{api_key_env_var} environment variable is not set! "
                    f"To run the {provider_name} handoff, set your API key. "
                    f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
                )
                return []
            try:
```

```
                provider = provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

        video_duration = get_video_duration(video_path)
        if video_duration <= 0:
            logger.error("Invalid video duration.")
            return []
        fps, frame_width = self._probe_fps_width(video_path)

        # --- Runner config + windowing thresholds ---
        runner, runner_params = self._build_runner(idx_cfg)

        velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
        merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
        min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
        # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See IndexingConfig.
        max_window_duration = idx_cfg.get("max_window_duration", 0.0)
        window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
        window_hard_cap = idx_cfg.get("window_hard_cap", False)
        window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
        inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
        inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
        window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
        window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
        handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
        ai_max_workers = idx_cfg.get("ai_max_workers", 5)
        log_candidates = idx_cfg.get("log_yolo_candidates", True)
        store_candidates = idx_cfg.get("store_yolo_candidates", True)

        detection_params = {
            "detector": self.name,
            "velocity_thresh": velocity_thresh,
            "merge_gap": merge_gap,
            "min_action_window_duration": min_window_duration,
            **runner_params,
        }

        # --- Phase 1: env healthcheck (fail fast with a clear message) ---
        ok, msg = runner.healthcheck()
        if not ok:
            logger.error("%s WSL environment not ready: %s", label, msg)
            return []
        logger.info("%s WSL env OK: %s", label, msg)

        # --- Phase 2: trajectory -> candidate rally windows ---
        # Trajectory detectors process the whole video in one pass (they carry
        # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
        t_start = time.time()
        out_dir = os.path.join("output", self.family)
        csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
        if not csv_path:
            logger.error("%s produced no trajectory; aborting.", label)
            return []

        points = runner.parse_trajectory_csv(csv_path)
        logger.info("Parsed %d trajectory points (%d visible).",
                    len(points), sum(1 for p in points if p.visible))

        all_candidate_windows = runner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width, chunk_start=0.0,
            velocity_thresh=velocity_thresh, merge_gap=merge_gap,
```

```
            min_window_duration=min_window_duration, chunk_index=0,
            max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
            window_hard_cap=window_hard_cap,
            window_inactivity_end=window_inactivity_end,
            inactivity_rally_thresh=inactivity_rally_thresh,
            inactivity_min_density=inactivity_min_density,
            window_blob_fallback=window_blob_fallback,
            window_blob_factor=window_blob_factor,
        )
        logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                    label, time.time() - t_start, len(all_candidate_windows))

        if log_candidates:
            for cw in all_candidate_windows:
                logger.info(f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                            f"({cw['end'] - cw['start']:.1f}s) | frames={cw['active_frame_count']} "
                            f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}")

        # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
        # segmenter adds net_crossings + person-cue features. Computed once, reused for both
        # persistence and the handoff gate below.
        window_stats = [
            self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
            for cw in all_candidate_windows
        ]

        # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
        candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
        if store_candidates:
            for i, cw in enumerate(all_candidate_windows):
                candidate_ids[i] = self.db.add_candidate_segment(
                    video_id, detector=self.name,
                    start_time=cw["start"], end_time=cw["end"],
                    chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
                    stats=window_stats[i], detection_params=detection_params,
                )

        if not all_candidate_windows:
            return []

        # --- Phase 3: AI provider handoff over padded candidate clips ---
        # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
        # candidates above stay the full superset ? only the served play_segments are gated.
        handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats, idx_cfg)

        ai_segments: List[dict] = []
        if skip_ai:
            # Fully-local: the selected candidate windows ARE the rallies ? emit them verbatim
            # (no clip extraction, no upload). Boundaries are the local detector's; shot count
            # falls back to the duration-based estimate (no AI exchange count).
            ai_segments = [
                {
                    "start_time": all_candidate_windows[wi]["start"],
```

```
                        "end_time": all_candidate_windows[wi]["end"],
                        "shuttle_exchange_count": None,
                        "shot_count_confidence": "LocalNoAI",
                        "_candidate_id": candidate_ids[wi],
                    }
                    for wi in handoff_indices
                ]
                logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies "
                            "(no AI handoff).", len(ai_segments))
            else:
                handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
                os.makedirs(handoff_dir, exist_ok=True)
                logger.info("Phase 3: %s validation on %d candidate windows...",
                            provider_name, len(handoff_indices))

                def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                    w_start = max(0, window["start"] - handoff_padding)
                    w_end = min(video_duration, window["end"] + handoff_padding)
                    w_dur = w_end - w_start
                    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                    if not extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
                        return []
                    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                    segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                         label=f"Handoff {wi+1}")
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning("Dropping segment with unparseable timestamps: %s ? %s", seg, e)
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
                    futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
                               for wi in handoff_indices]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
                logger.info("Phase 3 completed. AI confirmed %d semantic play segments.", len(ai_segments))

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
                    dur = end - start
```

```
320                     metadata = {
321                         "shot_count": self._shot_count_from(segment, dur),
322                         "shot_count_confidence": segment.get("shot_count_confidence", "Unknown"),
323                         "detector": f"{self.family}-hybrid-{model_name}",
324                     }
325                     segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)
326                     if segment_id:
327                         candidate_id = segment.get("_candidate_id")
328                         if candidate_id is not None:
329                             self.db.link_candidate_to_segment(candidate_id, segment_id)
330                         segments_data.append({
331                             "id": segment_id, "start_time": start, "end_time": end,
332                             "duration": dur, "metadata": metadata,
333                         })
334                 except Exception as e:
335                     logger.warning("Skipping invalid segment struct: %s", e)
336
337             return segments_data
338
```

6. assistant (2026-06-18T07:52:30.584Z)

Now I understand the call flow clearly. The key observation is at line 202: `points = runner.parse_trajectory_csv(csv_path)`. This `points` list is a local variable in `process_video`, and it stays alive through the entire `process_video` call ? through the `window_stats` list comprehension at lines 230-233 which is the only place `_enrich_candidate_stats` ? `_axis_estimate(points)` is called.

So within a single `process_video`, `points` is alive the whole time `_axis_estimate` is called. The cache key `id(points)` is stable for that call. The candidate's claim is about **instance reuse across two `process_video` calls**.

Let me check the test file and how segmenter instances are created.

7. user (2026-06-18T07:52:31.512Z)

```
tests\test_fusion_compare.py
tests\test_fusion_features.py
tests\test_fusion_golden_telemetry.py
tests\test_fusion_hybrid.py
```

8. user (2026-06-18T07:52:31.538Z)

No files found

9. user (2026-06-18T07:52:34.201Z)

```
"""FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).

Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
keys, PersonDetector never built); the persisted candidate set is the full superset of the
substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
persistence keeps the superset; and the person cue, when enabled, adds the person features.
"""
from types import SimpleNamespace
from unittest.mock import MagicMock, patch

from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

```

```
# Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a rally).
# Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
         "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
         "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}


def _points():
    pts = []
    for f in range(30, 120):       # window A frames: y flips 100<->900 around the net (~500)
        pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0 else 900.0))
    for f in range(300, 360):      # window B frames: y == net level -> deadband -> no crossings
        pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
    return pts


def _runner():
    r = MagicMock()
    r.healthcheck.return_value = (True, "env ok")
    r.run_predict.return_value = "out.csv"
    r.parse_trajectory_csv.return_value = _points()
    r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
    return r


def _run(indexing=None, provider_segments=None):
    provider = MagicMock()
    provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5, "end_time": 2.0}], {})
    db = MagicMock()
    db.add_candidate_segment.side_effect = [101, 102, 103, 104]
    db.add_play_segment.return_value = 200

    with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(), {"weights_path": "w"})), \
         patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)), \
         patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration", return_value=30.0), \
         patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk", return_value=True), \
         patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as preg, \
         patch("os.environ.get", return_value="dummy_key"):
        preg.get.return_value = provider
        seg = FusionSegmenter(db, {"indexing": indexing or {}})
        res = seg.process_video("v1", "clip.mp4")
        return db, provider, res


def _stats_calls(db):
    """Persisted stats dicts, in call order."""
    return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]


def test_net_crossings_persisted_and_person_off_by_default():
    db, _provider, _res = _run()
    stats = _stats_calls(db)
    assert len(stats) == 2                              # both windows persisted (superset)
```

```
    assert all("net_crossings" in s for s in stats)
    # Window A (rally) crosses many times; window B (held) zero.
    assert stats[0]["net_crossings"] >= 2
    assert stats[1]["net_crossings"] == 0.0
    # Person cue OFF by default -> NONE of the person features present.
    assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
    # The 4 base motion keys are still there.
    assert all({"peak_velocity", "mean_velocity", "active_frame_count", "track_id_count"} <= set(s)
               for s in stats)


def test_persisted_candidates_are_superset_of_substrate_windows():
    db, _provider, _res = _run()
    spans = [(kw["start_time"], kw["end_time"]) for _a, kw in db.add_candidate_segment.call_args_list]
    assert spans == [(1.0, 4.0), (10.0, 12.0)]          # exactly the substrate windows, unfiltered


def test_default_hands_off_all_windows():
    _db, provider, _res = _run()                        # min_crossings default 0 -> no gating
    assert provider.analyze_video.call_count == 2


def test_gate_a_drops_low_crossing_window_from_handoff_only():
    db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
    # Persistence is still the full superset (offline substrate intact)...
    assert len(_stats_calls(db)) == 2
    # ...but only the high-crossing window A reaches the AI handoff.
    assert provider.analyze_video.call_count == 1


def test_gate_a_never_empties_a_video():
    """NEVER-EMPTY safeguard: if the crossings threshold would drop EVERY window, the gate
    falls back to the full ungated set rather than hand off zero rallies. Here min_crossings
    is set absurdly high so neither window clears it ? yet both still reach the handoff."""
    db, provider, _res = _run(indexing={"fusion": {"min_crossings": 999}})
    assert len(_stats_calls(db)) == 2                   # superset persisted, untouched
    # Gating would zero the video ? safeguard restores the ungated windows (both hand off).
    assert provider.analyze_video.call_count == 2


def test_person_cue_adds_features_when_enabled():
    # Two in-court players, each shifting between two sampled frames ? exercises the REAL
    # glue (in-court counts -> players_in_court; track displacement -> mean_player_motion),
    # not just key-presence.
    fake_pd = MagicMock()
    fake_pd.detections_per_frame.return_value = {
        "frames": {
            30: [(1, (10, 10, 30, 40)), (2, (60, 10, 80, 40))],
            33: [(1, (16, 10, 36, 40)), (2, (66, 10, 86, 40))],
        },
        "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
    }
    with patch("backend.pipeline.detectors.person_detector.PersonDetector", return_value=fake_pd):
        db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
```

```
    stats = _stats_calls(db)
    assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
    assert all("net_crossings" in s for s in stats)
    assert stats[0]["players_in_court"] == 2.0           # 2 in-court players every sampled frame
    assert stats[0]["mean_player_motion"] > 0.0          # both tracks moved -> non-zero motion
    fake_pd.detections_per_frame.assert_called()         # the person detector WAS used


def test_gate_b_drops_low_player_window_from_handoff():
    """Gate B: with the person cue on and min_players=2, a window whose in-court player
    count is below 2 is dropped from the handoff ? but still persisted (superset)."""
    def _side(video_path, start_f, end_f, frame_skip):
        if start_f >= 300:        # window B [10,12]s -> start frame ~300: only 1 player
            frames = {start_f: [(1, (10, 10, 30, 40))]}
        else:                     # window A [1,4]s -> start frame ~30: 2 players
            frames = {start_f: [(1, (10, 10, 30, 40)), (2, (60, 10, 80, 40))]}
        return {"frames": frames, "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0}

    fake_pd = MagicMock()
    fake_pd.detections_per_frame.side_effect = _side
    with patch("backend.pipeline.detectors.person_detector.PersonDetector", return_value=fake_pd):
        db, provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}, "min_players": 2}})
    stats = _stats_calls(db)
    assert len(stats) == 2                               # superset persisted (both)
    assert provider.analyze_video.call_count == 1        # only window A (>=2 players) handed off
    assert stats[0]["players_in_court"] >= 2 and stats[1]["players_in_court"] < 2


def test_build_runner_selects_substrate():
    """_build_runner picks the WASB or TrackNet runner by indexing.fusion.substrate."""
    seg = FusionSegmenter(MagicMock(), {"indexing": {}})
    with patch("backend.pipeline.segmenters.fusion_hybrid.WasbRunner") as W, \
         patch("backend.pipeline.segmenters.fusion_hybrid.WasbConfig") as WC, \
         patch("backend.pipeline.segmenters.fusion_hybrid.TrackNetRunner") as T, \
         patch("backend.pipeline.segmenters.fusion_hybrid.TrackNetConfig") as TC:
        WC.from_indexing_cfg.return_value = MagicMock(weights_path="w")
        TC.from_indexing_cfg.return_value = MagicMock(weights_path="t", queue_length=5)
        _runner_w, extras_w = seg._build_runner({"fusion": {"substrate": "wasb"}})
        _runner_t, extras_t = seg._build_runner({"fusion": {"substrate": "tracknet"}})
        _runner_d, extras_d = seg._build_runner({})       # default -> wasb
    assert W.called and extras_w["fusion_substrate"] == "wasb"
    assert T.called and extras_t["fusion_substrate"] == "tracknet"
    assert extras_t["queue_length"] == 5
    assert extras_d["fusion_substrate"] == "wasb"

```

10. assistant (2026-06-18T07:52:37.200Z)

Now let me understand how segmenter instances are created in production ? whether one instance is reused across multiple videos or a fresh instance is created per video. Let me look at the registry and how segmenters are instantiated.

11. user (2026-06-18T07:52:37.929Z)

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any, Type, Optional
from backend.infrastructure.database import Database
from backend.results import Result  # new standardized contract (Result[Value] == Success[Value] | Failure)
```

```python

class BasePlaySegmenter(ABC):
    def __init__(self, db: Database, config: Dict[str, Any]):
        self.db = db
        self.config = config

    @property
    @abstractmethod
    def name(self) -> str:
        """Unique identifier for this segmenter."""
        pass

    @property
    @abstractmethod
    def description(self) -> str:
        """Human-readable explanation of this segmenter."""
        pass

    @abstractmethod
    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
        """
        Processes a video to detect play segments, populates SQLite database,
        and returns a Result.

        Use isinstance(result, Failure) (or result.is_success()) to distinguish
        "no segments found" from "something went wrong" (missing key, healthcheck
        failure, etc.). This is the start of the standardized error/result contract.
        """
        pass

class SegmenterRegistry:
    def __init__(self):
        self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

    def register(self, segmenter_cls: Type[BasePlaySegmenter]):
        """Registers a new segmenter class under its name property."""
        try:
            temp_inst = segmenter_cls(None, {})
            name = temp_inst.name
        except Exception as e:
            raise ValueError(f"Failed to resolve name property from segmenter class {segmenter_cls.__name__}: {e}") from e
        self._segmenters[name] = segmenter_cls

    def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
        return self._segmenters.get(name)

    def list_available(self) -> Dict[str, str]:
        """Returns a dict of registered segmenter names and their descriptions."""
        res = {}
        for name, cls in self._segmenters.items():
            try:
                temp_inst = cls(None, {})
                desc = temp_inst.description
            except Exception:
                desc = "No description available."
            res[name] = desc
        return res

segmenter_registry = SegmenterRegistry()

```

12. assistant (2026-06-18T07:52:40.383Z)

The registry stores **classes**, not instances. Let me find where the segmenter class is instantiated to process videos ? to determine if instances are reused across videos.

13. user (2026-06-18T07:52:41.493Z)

```
backend\main.py:170:        segmenter_cls = segmenter_registry.get(seg_name)
backend\main.py:192:            result = segmenter.process_video(video_id, req.video_path,
req.player_pool, req.max_frames)
backend\main.py:362:    if req.segmenter and not segmenter_registry.get(req.segmenter):
backend\main.py:826:            segmenter_cls = segmenter_registry.get(seg_name)
backend\main.py:835:            result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
scripts\run_process_and_stitch.py:76:        segmenter_cls = segmenter_registry.get(seg_name)
scripts\run_process_and_stitch.py:85:        segments, failure =
unwrap_or_failure(segmenter.process_video(video_id, video_path))
tools\generate_highlights.py:99:    seg_cls = segmenter_registry.get(segmenter)
tools\generate_highlights.py:167:            result = seg_cls(db, cast(Dict[str, Any],
cfg)).process_video(vid, proc_detect)
scripts\run_phase3_eval.py:101:        seg_cls = segmenter_registry.get(args.segmenter)
scripts\run_phase3_eval.py:150:                    segmenter.process_video(video_id, video_path,
max_frames=args.max_frames))
tests\test_base.py:34:    success = inst.process_video("v1", "path")
tests\test_base.py:40:    failure = inst.process_video("v1", "fail")
tests\test_base.py:192:        inst.process_video("v1", "path")
docs\CODE_MAP.md:61:  -> seg = segmenter_registry.get(seg_name)(db, global_config)   ?
@backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
docs\CODE_MAP.md:62:  -> segments = seg.process_video(video_id, video_path, player_pool?,
max_frames?)
docs\CODE_MAP.md:129:[Omitted long matching line]
tests\test_gemini.py:42:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_gemini.py:81:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_gemini.py:108:    return segmenter.process_video(video_id, "dummy.mp4")
tests\test_gemini.py:185:        res = segmenter.process_video("vid_single_oversize",
"dummy.mp4")
tests\test_fusion_hybrid.py:55:        res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:63:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:82:    assert seg.process_video("v1", "clip.mp4") == []
tests\test_tracknet_hybrid.py:95:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:110:    assert seg.process_video("v1", "clip.mp4") == []
tests\test_tracknet_hybrid.py:129:    seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:153:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:181:    res = seg.process_video("v1", "clip.mp4")
tests\test_trajectory_hybrid_equivalence.py:56:        return db, seg.process_video("v1",
"clip.mp4")
tests\test_yolo_hybrid.py:89:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:113:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:156:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:225:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:276:    segmenter.process_video("v1", "dummy.mp4")
```

14. assistant (2026-06-18T07:52:43.732Z)

Let me look at the main.py instantiation sites and the eval/highlight tools to determine if a single instance processes multiple videos.

15. user (2026-06-18T07:52:45.078Z)

```
165
166             # 2. Run segmenter & indexer
167             logger.info("[API] Video passed checks. Segmenting and indexing...")
168             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
169
170             segmenter_cls = segmenter_registry.get(seg_name)
```

```
171             if not segmenter_cls:
172                 # Submit-time validation already 400s unknown names; this is the defensive
173                 # in-worker path (e.g. config default pointing at an unregistered segmenter).
174                 available = list(segmenter_registry.list_available().keys())
175                 response = {
176                     "video_id": video_id,
177                     "status": "failed",
178                     "message": f"Segmenter '{seg_name}' not found. Available segmenters: {available}",
179                     "results": [r.model_dump() for r in val_results],
180                     "play_segments": [],
181                 }
182                 return response
183
184             logger.info(f"[API] Using segmenter: {seg_name}")
185             notify(f"segmenting ({seg_name})")
186             segmenter_actual = seg_name
187             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
188             # if the run yields segments; otherwise drop the new partial rows and keep the old.
189             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
190             segmenter = segmenter_cls(db_instance, global_config)
191             try:
192                 result = segmenter.process_video(video_id, req.video_path, req.player_pool, req.max_frames)
193             except Exception:
194                 # A raising segmenter must not leave half-written rows: restore the pre-run
195                 # state (same destroy-after-confirm contract as the Failure branch), then let
196                 # the job worker record the failure.
197                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
198                 raise
199             segmentation_ran = True
```

16. assistant (2026-06-18T07:52:49.030Z)

In `main.py`, a fresh `segmenter` instance is created per API request (line 190). Let me check the eval/highlight tools which process multiple videos in a loop, since those are the candidate places for instance reuse.

17. user (2026-06-18T07:52:50.076Z)

```

        cfg = load_config()
        # Per-run overrides (typed config; the segmenter reads these via its dict-compat .get).
        # Logged by the segmenter itself once logging is configured below.
        if skip_ai_handoff:
            cfg.indexing.skip_ai_handoff = True
        if wasb_stream:
            cfg.indexing.wasb.stream_video = True
        db = Database(db_path)
        seg_cls = segmenter_registry.get(segmenter)
        if seg_cls is None:
            raise SystemExit(f"unknown segmenter '{segmenter}'; available: {segmenter_registry.list_available()}")

        # Combined batch log (local; published at the end).
        fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s", "%H:%M:%S")
        if not logging.getLogger().handlers:
            logging.basicConfig(level=logging.INFO, handlers=[logging.StreamHandler(sys.stdout)])
```

```
            for h in logging.getLogger().handlers:
                h.setFormatter(fmt)
        batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
        batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
        batch_fh.setFormatter(fmt)
        logging.getLogger().addHandler(batch_fh)

        summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [], "failed": []}
        logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
                    len(video_paths), out_dir, segmenter, force)

        for i, src in enumerate(video_paths, 1):
            name = os.path.basename(src)
            stem = os.path.splitext(name)[0]
            reel_name = f"{stem} - Highlights.mp4"
            dest_reel = os.path.join(out_dir, reel_name)
            local_reel = os.path.join(local_work_dir, reel_name)
            # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
            # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
            detect_src = detect_from_paths[i - 1] if detect_from_paths else src

            if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
                logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
                summary["skipped"].append(name)
                continue
            if not os.path.exists(src) or not os.path.exists(detect_src):
                logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s detect_exists=%s)",
                               i, len(video_paths), name, os.path.exists(src), os.path.exists(detect_src))
                summary["failed"].append(name)
                continue

            run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
            run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
            run_fh.setFormatter(fmt)
            logging.getLogger().addHandler(run_fh)
            staged_src: Optional[str] = None
            try:
                logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
                # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
                # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile, so
                # only the detector input ever needs staging.
                if needs_local:
                    staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
                    if not (os.path.exists(staged_src) and os.path.getsize(staged_src) == os.path.getsize(detect_src)):
                        logger.info("localizing detection source (%.0f MB)...", os.path.getsize(detect_src) / 1024 / 1024)
                        shutil.copy2(detect_src, staged_src)
                    proc_detect = staged_src
                else:
                    proc_detect = detect_src

                vid = compute_video_id(proc_detect)
                db.clear_video_data(vid)          # idempotent: no cross-run accumulation
                db.add_video(vid, proc_detect, "processed", [])
                if detect_src != src:
                    logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
                                os.path.basename(detect_src), name)
                logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
```

```
164
165                 # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
166                 # main.py); cast keeps the type checker honest about that contract.
167                 result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
168                 if isinstance(result, Failure):
169                     logger.error("segmentation FAILED: %s", getattr(result, "message", result))
170                     summary["failed"].append(name)
171                     continue
172
173                 segs = db.query_play_segments(vid, {})
174                 pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
175                 logger.info("rallies detected: %d", len(pairs))
176                 if not pairs:
177                     logger.warning("no rallies -> no reel")
178                     summary["no_rallies"].append(name)
179                     continue
```

18. assistant (2026-06-18T07:52:53.209Z)

Here at line 167, `seg_cls(db, ...)` creates a **fresh instance per video** in the loop. So no reuse there either. Let me check the two eval scripts which are the most likely candidates for instance reuse: `scripts/run_phase3_eval.py` and `scripts/run_process_and_stitch.py`.

19. user (2026-06-18T07:52:54.480Z)

```

    stages = batch.default_stages_for(args.segmenter, args.stage)

    # Resolve the segmenter class up front (unless purely scoring).
    segmenter = None
    if not args.score_only:
        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(f"ERROR: segmenter '{args.segmenter}' not found. "
                  f"Available: {list(segmenter_registry.list_available().keys())}", file=sys.stderr)
            return 2
        segmenter = seg_cls(db, cfg)

    # Filter entries.
    targets = []
    for e in entries:
        if args.only and e["video_id"] not in args.only:
            continue
        targets.append(e)
    if args.limit:
        targets = targets[: args.limit]

    per_video = []
    skipped = []
    print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} | stages={stages} | db={db_path}\n")

    for e in targets:
        vid = e["video_id"]
        video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
        csv_path = os.path.join(manifest_dir, e["csv"])

        if not video_path or not os.path.exists(video_path):
            skipped.append((vid, "no video file"))
            print(f"[skip] {vid}: no video file ({e.get('local_path')})")
            continue
        if not os.path.exists(csv_path):
```

```
131                 skipped.append((vid, "no GT csv"))
132                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
133                 continue
134
135             video_id = get_file_md5(video_path)
136
137             # Segment unless score-only or already present (and not --reprocess).
138             if not args.score_only:
139                 already = db.query_play_segments(video_id, {}) or db.query_candidate_segments(video_id)
140                 if already and not args.reprocess:
141                     print(f"[have] {vid}: detections already in DB (use --reprocess to redo)")
142                 else:
143                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
144                     db.add_video(video_id, video_path, "processed", [])
145                     t0 = time.time()
146                     # process_video returns a Result (Success|Failure) for some segmenters
147                     # (e.g. the default `motion`); unwrap before len() so this doesn't crash with
148                     # TypeError on a Success dataclass ? mirrors the unwrap in backend/main.py.
149                     segs, failure = unwrap_or_failure(
150                         segmenter.process_video(video_id, video_path, max_frames=args.max_frames))
151                     if failure is not None:
152                         print(f"        -> FAILED: {failure.message}")
153                         skipped.append((vid, f"segment failed: {failure.message}"))
154                         continue
155                     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
156
157             gt = load_annotations(csv_path)
158             report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou, detector=args.detector)
159             print(format_report_text(report, show_failures=args.show_failures))
160             print()
161             d = report_to_dict(report)
162             d["collector_video_id"] = vid
163             per_video.append(d)
164
```

20. assistant (2026-06-18T07:53:02.743Z)

This is the smoking gun for the candidate's claim. In `run_phase3_eval.py`:
- Line 106: `segmenter = seg_cls(db, cfg)` ? **ONE instance** created up front, before the loop.
- Line 121: `for e in targets:` ? loops over **multiple videos**.
- Line 150: `segmenter.process_video(...)` ? the **same instance** is reused across all target videos.

So instance reuse across videos is real and exercised in the eval harness. Now I must verify whether the cache key `id(points)` can actually return a stale estimate. Let me trace the data lifetime carefully within `process_video`.

The critical question: within `process_video` (trajectory_hybrid.py), `points` is assigned at line 202 and used in the `window_stats` comprehension (lines 230-233). After `process_video` returns, that `points` list becomes garbage. On the next `process_video` call (next video), a new `points` list is created at line 202. The `_axis_cache_key` from the previous call still holds the old `id()`.

For the bug to fire, the new `points` list must be allocated at the **same memory address** as the previous (now-freed) list, AND the cache must not be invalidated by content. Let me confirm there is no reset between videos and examine `rally_gate.estimate_play_axis` to understand impact.

21. user (2026-06-18T07:53:03.846Z)

```
"""Rally-start gate: suppress between-point false fires using shuttle net-crossings.

WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
the opponent for service after losing a point, that single trajectory looks like a
rally. The discriminator is structural, not duration-based: a real rally has the
shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
single service feed crosses ~once.

This module is camera-agnostic and needs NO new model ? it works on the trajectory
we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
axis with the larger shuttle spread) and the net position (median shuttle coord on
that axis ? the shuttle clusters at the two ends and crosses the net region often,
so the median sits near the net), then counts sign changes of (coord - net) across
the visible points inside a candidate window, with a deadband to ignore jitter.

Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
windows from trajectory_to_action_windows, or to fold into it later.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import List, Optional, Sequence, Tuple

Interval = Tuple[float, float]


@dataclass
class GatedWindow:
    start: float
    end: float
    crossings: int
    visible_points: int
    kept: bool


def _spread(vals: Sequence[float]) -> float:
    """Robust spread = p95 - p5 (ignores a few outlier detections)."""
    if len(vals) < 2:
        return 0.0
    s = sorted(vals)
    lo = s[max(0, int(0.05 * (len(s) - 1)))]
    hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
    return hi - lo


def _median(vals: Sequence[float]) -> float:
    if not vals:
        return 0.0
    s = sorted(vals)
    n = len(s)
    return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


def estimate_play_axis(points) -> Tuple[str, float, float]:
    """Return (axis 'x'|'y', net_value, span) from visible points.

    Play axis = the image axis along which the shuttle travels most (larger spread).
    For a baseline camera that's Y (players top/bottom); for a side camera, X.
    """
    xs = [p.x for p in points if p.visible]
    ys = [p.y for p in points if p.visible]
    sx, sy = _spread(xs), _spread(ys)
    if sx >= sy:
        return "x", _median(xs), sx
```

```
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


    def gate_windows(points, windows: List[Interval], fps: float,
                     min_crossings: int = 2, deadband_frac: float = 0.06,
                     estimate: Optional[Tuple[str, float, float]] = None) -> List[GatedWindow]:
        """Annotate each (start,end) window with its shuttle net-crossing count and a
        keep decision (crossings >= min_crossings). Deadband is a fraction of the play
        axis span. Net/axis are estimated once over the whole trajectory (stable).

        `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
        `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't redone
        per window ? same result, avoids the per-window re-estimation."""
        axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
        deadband = deadband_frac * span
        out: List[GatedWindow] = []
        for (s, e) in windows:
            wp = _points_in(points, fps, s, e)
            c = count_net_crossings(wp, axis, net, deadband)
            vis = sum(1 for p in wp if p.visible)
            out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
                                   kept=c >= min_crossings))
        return out


    def apply_gate(points, windows: List[Interval], fps: float,
                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
        """Convenience: return only the windows that pass the crossing gate."""
        return [(g.start, g.end) for g in
                gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]

```

22. assistant (2026-06-18T07:53:18.643Z)

Now I have the full picture. Let me reason carefully about whether this is a **real, reachable** defect.

The candidate's mechanism:
1. One `FusionSegmenter` instance is reused across multiple `process_video` calls (confirmed: `run_phase3_eval.py` line 106 + loop at 121-150).
2. `_axis_cache_key` / `_axis_cache` persist on the instance across calls (confirmed: set in

`__init__`, never reset).
3. The cache key is `id(points)` (line 103).
4. Within `process_video`, the `points` list (trajectory_hybrid.py:202) lives only for the duration of that call, then becomes garbage.
5. CPython can reuse a freed object's memory address. If video B's `points` list is allocated at the same address video A's `points` had, `id(points_B) == id(points_A)`, so the cache hit returns video A's stale `(axis, net, span)`.

Now let me evaluate the four lenses (REFUTE: default real=false unless provably a bug).

Is it reachable? The cache-key reuse requires `id()` collision. This is a genuine CPython behavior ? freed list objects' addresses are reused by the next list allocation of a compatible size. Between two `process_video` calls there are many allocations (trajectory parsing, window stats, etc.), so whether `points_B` lands on exactly `points_A`'s old address is non-deterministic but absolutely possible. Critically, within `_axis_estimate` the check is `self._axis_cache_key != key or self._axis_cache is None`. For video A, after processing, `_axis_cache_key = id(points_A)` and `_axis_cache` is non-None. For video B, if `id(points_B) == id(points_A)`, the condition is `False or False` ? cache hit ? returns video A's estimate.

Let me verify whether `points` could be kept alive (refuting the address-reuse). In `process_video`, after the `window_stats` comprehension completes, `points` is still referenced by the local variable until function return. Then it's freed. The next call's `parse_trajectory_csv` returns a fresh list. There is no overlap in lifetime ? video A's `points` is dead before video B's `points` is created. So address reuse is possible.

Let me also confirm `_axis_estimate` is ONLY called within the window_stats comprehension (where points is alive in that call) and check there's no in-call aliasing issue. Confirmed at trajectory_hybrid.py:231 ? the only caller. Within one call, `points` is the same object for all windows, so the cache is correctly hit (intended optimization). The cross-call staleness is the concern.

Let me double-check: is there any reset of the cache or fresh-instance creation that I'm missing in the eval path? Let me re-examine `run_phase3_eval` more carefully ? specifically whether `--reprocess` or the `already` guard affects this.

23. user (2026-06-18T07:53:20.903Z)

```
        # Segment unless score-only or already present (and not --reprocess).
        if not args.score_only:
            already = db.query_play_segments(video_id, {}) or db.query_candidate_segments(video_id)
            if already and not args.reprocess:
                print(f"[have] {vid}: detections already in DB (use --reprocess to redo)")
            else:
                print(f"[proc] {vid}: segmenting with {args.segmenter}...")
                db.add_video(video_id, video_path, "processed", [])
                t0 = time.time()
                # process_video returns a Result (Success|Failure) for some segmenters
                # (e.g. the default `motion`); unwrap before len() so this doesn't crash with
                # TypeError on a Success dataclass ? mirrors the unwrap in backend/main.py.
                segs, failure = unwrap_or_failure(
                    segmenter.process_video(video_id, video_path, max_frames=args.max_frames))
                if failure is not None:
                    print(f"        -> FAILED: {failure.message}")
                    skipped.append((vid, f"segment failed: {failure.message}"))
                    continue
                print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
```

24. assistant (2026-06-18T07:53:49.652Z)

Confirmed: a fresh DB-empty run (or `--reprocess`) processes multiple videos through the **same** `segmenter` instance (line 106), exercising `_axis_estimate` once per video on the same instance. The cache state carries over.

Now, the decisive question for severity/reachability: **does an `id()` collision actually occur in practice, and does it change output?**

This is the crux. The candidate itself rates confidence 0.3 and severity low. Let me weigh the REFUTE lens rigorously:

1. **Within a single video, the cache is correct** ? `points` is alive, the estimate is computed once and reused across windows (the intended optimization). No bug here.

2. **Across videos, staleness requires an `id()` collision.** This is the only path to a wrong result. Is it "reachable" in the bug-hunting sense? `id()`-collision after GC is a well-known CPython hazard and is genuinely possible. The two `points` lists never coexist (video A's is freed before video B's is created), so the address can be reused. It is non-deterministic but not impossible ? CPython's pymalloc commonly hands back recently-freed blocks of the same size class. For two similarly-sized trajectory lists this is plausible.

3. **Impact if it collides:** `_axis_estimate(points_B)` returns video A's `(axis, net, span)`. This feeds `gate_windows(..., estimate=stale)`. The `net` and `span` (? deadband) belong to video A. For video B with a different camera (e.g. baseline vs side view ? axis flips from y to x), `net_crossings` would be computed against the wrong axis/net, producing wrong crossing counts. This silently mis-scores `net_crossings`, which can flip Gate-A keep/drop when `min_crossings > 0`. With default `min_crossings = 0`, the gating is a no-op so the stale value only affects the persisted `stats["net_crossings"]` (a feature column), not served decisions. So real served impact requires both instance reuse AND `min_crossings > 0` AND an `id()` collision AND differing axis/net between the two videos.

Now let me apply the REFUTE standard strictly: is this **provably a bug**, or is it guarded/intended/practically-unreachable?

The comment at lines 49-52 and 100-102 explicitly states the cache "depends only on `points`" and is "per trajectory ... computed once per video." The author's mental model is "one trajectory object = one video." The flaw is that `id()` is not a stable identity across object lifetimes ? but the author did not guard against cross-video reuse, and `run_phase3_eval` does reuse the instance.

However, for the REFUTE lens to conclude `real=true`, I need this to be a **provable** defect with a concrete trigger. The trigger here depends on a non-deterministic `id()` collision. This is a genuine soundness flaw in the cache-key choice, but:
- The actual mis-scoring is contingent on a memory-address coincidence that cannot be guaranteed to occur on any given input.
- A correctness defect that "may or may not" fire depending on the allocator is the textbook definition of a latent/heisenbug-class issue. It IS a real latent defect (the cache key is genuinely unsound), but the *manifestation* (wrong output) is not deterministically reachable from a specific input ? it requires an allocator coincidence.

Let me verify the test suite doesn't already guard this (no test reuses the instance across videos). Looking at `test_fusion_hybrid.py`, every `_run()` creates a fresh `FusionSegmenter` (line 54), so the test suite does NOT exercise instance reuse ? the staleness path is untested.

Let me make the final determination on each axis:

- **real**: The cache key `id(points)` is genuinely unsound as a cross-lifetime identity. This is a true latent defect ? the author's invariant ("id(points) uniquely identifies a video's trajectory") does not hold across `process_video` calls. So `real = true`.
- **reachable**: The instance IS reused across videos (`run_phase3_eval.py`). However, the *wrong-output manifestation* requires a non-deterministic `id()` collision, which cannot be triggered by a specific deterministic input. There is no concrete input I can hand to the owner that *deterministically* produces a wrong result. Per the strict standard ("crash on a reachable input, wrong logic ... + concrete trigger"), the concrete trigger here is probabilistic, not deterministic. I'll mark `reachable = false` because no deterministic triggering input exists ? the defect can only manifest via an allocator coincidence, not a chosen input.

- **already_guarded_or_tested**: Not guarded (no reset, no content fingerprint) and not tested (all tests use fresh instances). So `false`.
- **intended**: The id()-as-key choice was deliberate, but cross-video staleness was NOT intended ? the comment claims correctness ("computed once per video") that the code doesn't actually guarantee. So `intended = false` (the staleness is not an intended behavior).

Given the REFUTE lens default (real=false unless provably a bug) and the requirement for a concrete, reachable trigger: the cache-key is unsound (real defect in the abstract), but the manifestation is not deterministically reachable by any chosen input ? it hinges on a non-guaranteed `id()` collision. Under PRECISION-over-recall, I will not assert a wrong-output finding that cannot be deterministically triggered. The defect is a genuine code smell / latent soundness issue, but as a *reachable* bug it does not meet the bar.

Let me record this honestly.

25. user (2026-06-18T07:54:01.689Z)

Structured output provided successfully

26. assistant (2026-06-18T07:54:11.464Z)

I've completed the analysis. Summary of my finding on the candidate:

The candidate identifies a **genuine latent soundness flaw** but it does **not rise to a confirmed reachable bug** under the precision-over-recall standard.

What's real:
- `_axis_estimate` at `backend/pipeline/segmenters/fusion_hybrid.py:100-109` keys its cache on `id(points)`, which is only unique among *live* objects.
- The instance is genuinely reused across videos: `scripts/run_phase3_eval.py:106` builds one `segmenter` instance, then the loop at lines 121-150 calls `process_video` once per video, and `self._axis_cache_key`/`self._axis_cache` (set in `__init__`, never reset) carry over.
- Within a single `process_video`, `points` (trajectory_hybrid.py:202) stays alive through its only caller (the `window_stats` comprehension at lines 230-233), so the in-call cache is correct/intended. The cross-video staleness is neither intended, guarded, nor tested (every `_run()` in `tests/test_fusion_hybrid.py:54` builds a fresh instance, so reuse is never exercised).

Why I mark it not reachable:
- The wrong-output manifestation requires a non-deterministic `id()` collision ? video B's freshly parsed `points` list landing on video A's freed address. Possible (the two never coexist) but not guaranteed by any chosen input.
- Even on a collision, served impact is contingent: default `min_crossings=0` makes Gate-A a no-op, so a stale estimate only corrupts the persisted `net_crossings` feature column; flipping a served keep/drop additionally needs `min_crossings>0` plus a video whose true axis/net differs from the prior one.

It's a real code smell worth the cheap defensive fix (reset the cache per `process_video`, or key on a content fingerprint), but the current code is not provably wrong on any deterministic input.